VOID HACKS() 8.0 · TRACK: ABHEDYA — CYBER SECURITY & DIGITAL FORENSICS

Data Context & Data Contract

आँकड़ा संदर्भ

What the 2,000,000-row transaction dataset actually is, what every column means, how accounts become a graph, how collector / distributor / L1–L3 must be understood, why Transaction_ID is not a key, and the row-level provenance rule that makes every number in the system citable back to the source file.

VERSION	1.0 — data contract
STATUS	Frozen for v1.0
OWNER	Data & Backend
COMPANION TO	PRD v2.1 · Backend API v1.0 · Frontend UI v1.0
SUPERSEDES	PRD v2.1 data contract on Transaction_ID uniqueness
ASSOCIATED ORG	Indore Police Commissionerate
EVENT	Void Hacks() 8.0
DATE	02 October 2026

[PS] problem statement · [DEC] team decision · [ASSUME] assumption · [DATA-PRELIM] recalculate with the profiler

Version 1.0 · Read this before you touch the data

This document defines the data context for the ABHEDYA-CHAKRA cyber-financial investigation system. It is the contract that every other layer inherits: ingestion, preprocessing, graph construction, detection, risk scoring, the API, the investigation UI, evidence assembly and document generation all depend on the definitions here being followed exactly.

0.1 What this document settles

It defines	Section
What the transaction dataset contains	§2, §3, §5
What each column means and how it is normalized	§6
What derived features must be created	§10, §19
How accounts and transactions become a graph	§11–§14
How suspicious signals must be interpreted	§9, §24–§26
How L1 / L2 / L3 roles are to be understood	§15–§18
What assumptions are allowed, and what are forbidden	§4, §29, §30, §32
How evidence stays traceable to the original dataset	§27, §28, §40

0.2 Where this document sits

```
PRD v2.1                what the product must do and why
   │
   ├── DATA CONTEXT v1.0   <-- this document: what the data IS
   │                           and the rules for touching it
   │
   ├── Backend API v1.0     how the data is exposed (45 routes)
   │
   └── Frontend UI v1.0     how the data is shown
```

Where this document and the PRD disagree on a **data rule**, this document wins and the PRD is corrected. Where they disagree on **product behaviour**, the PRD wins.

0.3 Markers

Marker	Meaning
[PS]	From the official problem statement
[DEC]	Team engineering decision
[ASSUME]	Assumption not yet confirmed — see §29, §30 and the PRD's open questions
[DATA-PRELIM]	Observed in preliminary profiling, **not yet re-verified** by `make profile-check`
MUST / MUST NOT	A violation is a defect, not a preference

0.4 Verification status of every number in this document

Source of the figures below	Preliminary profiling of the supplied CSV

Independently recomputed for this document?	**No**
What makes them authoritative	`python3 tools/profile_dataset.py <csv> --check` → `data/artifacts/profile.json`
Until then	Every figure carries **[DATA-PRELIM]** and **MUST NOT** be hard-coded anywhere

> **The single most important sentence in this document:** the supplied CSV is the source of truth for exact counts — not the problem statement, not the PRD, and not this file. Every number here is an expectation to be checked, and the profiler is what checks it.

---

1. Purpose

This file is intended to be read **before** implementing any of:

```
ingestion  ·  preprocessing  ·  graph construction  ·  detection
risk scoring  ·  API endpoints  ·  investigation UI
evidence generation  ·  AI/document generation
```

It exists because most of the expensive mistakes available on this project are **data mistakes**, and almost all of them are irreversible by the time they are noticed:

Mistake	When it surfaces	Cost
Treating `Transaction_ID` as a primary key	When a citation in a freeze requisition points at the wrong transfer	Catastrophic — the document is wrong and the error is invisible
Converting an account number to an integer	When a leading-zero account silently becomes a different account	High — wrong account frozen
Deleting duplicate rows "to clean the data"	Never, visibly. The rows are simply gone	Catastrophic — evidence destroyed
Pass-through as `out_total / in_total`	During tuning, as inexplicably bad precision	Medium — wasted hours
Hard-coding a profiling figure	When the organisers hand over a different file	High — demo-day failure
Treating a device string as proof	In front of a judge who asks "so anyone on Linux is a criminal?"	Fatal to credibility

Each one has a rule in this document, and each rule has a test in the PRD's test matrix.

---

2. Dataset Identity

2.1 Source

```
FILENAME   VoidHacks8_MuleAccount_2M_Transactions.csv
FORMAT     UTF-8 CSV, header row, comma-delimited
SIZE       ~274 MB  [DATA-PRELIM]
LOCATION   data/raw/   (gitignored — never committed)
```

2.2 Expected scale

Property	Expected	Authority
Transaction rows	~2,000,000	[PS] and observation
Unique accounts	~24,873	observation **[DATA-PRELIM]**
Transaction window	15 days	[PS] and observation
Source columns	11	observation **[DATA-PRELIM]**

The official problem statement specifies **2,000,000 banking records across a 15-day period** and an evaluation population of **1,500 mule + 23,500 regular = 25,000 accounts** [PS]. The observed ~24,873 accounts is consistent with that population; it does not prove the two are the same set (§30).

2.3 The identity rule

The actual supplied CSV must always be treated as the source of truth for exact observed counts. [DEC]

```
PS says             ->  an expectation
This document says  ->  an expectation
profile.json says   ->  the truth for this file
```

If the organisers supply a second file, or a corrected file, nothing in the codebase should need editing: the profiler re-measures, `profile-check` re-asserts, and every threshold derived from a distribution is recomputed.

---

3. Actual Dataset Profile

Initial profiling of the supplied CSV produced approximately the following. **All values are [DATA-PRELIM].**

3.1 Shape and identity

Property	Observed
Transaction rows	2,000,000
Unique accounts	24,873
Unique `Transaction_ID` values	~1,997,748
Duplicate `Transaction_ID` row surplus	~2,252
Missing values	none observed
File size	~274 MB
Date range	2026-09-15 → 2026-09-29

3.2 Amounts

Property	Observed
Minimum	~₹50
Median	**~₹899.35**
Mean	~₹1,760.88
Maximum	~₹499,823.99

The median is the most consequential number in this table. Any rupee threshold in the system — trace pruning floors, freeze minimums, sink thresholds — must be calibrated against it. A flat ₹1,000 floor, which an earlier draft of the PRD used, sits above the median transaction and would silently truncate genuine money trails (§23, §31 rule 7).

3.3 Derived quantities worth knowing before you write code

Quantity	Value	Why it matters
Transactions per day	~133,333	Timeline bucket sizing
Avg transactions per account, as sender	**~80.4**	This is the out-degree, and it is large
Avg transactions per account, as receiver	**~80.4**	The in-degree is equally large — **every account does both** (§13)
Avg total degree	~160.8	
Expected unique (sender→receiver) pairs	**~1,996,771**	≈99.8% of rows are their own unique pair — edge aggregation buys almost nothing
Implied CSR size, both directions	~128 MB	Size the arrays for 2.0 M edges, not for a few hundred thousand

3.4 The rule

> **These values are initial profiling results. The ingestion pipeline MUST independently calculate and store the final dataset profile. Do not hard-code these numbers. MUST**

Concretely:

```
# WRONG — a number that will be wrong the moment the file changes
if row_count != 2_000_000:
    raise ValueError("bad dataset")

# RIGHT — expectations live in config, observations live in profile.json,
#         and the two are compared by an explicit, loggable check
EXPECTED_ROWS = 2_000_000          # config.py, documented as an expectation
profile = run_profiler(csv_path)   # measures reality
assert_profile(profile, config)    # reports every mismatch as a P0 finding
```

The runnable implementation of this is `tools/profile_dataset.py` (FR-A05). It writes `data/artifacts/profile.json` and, with `--check`, exits non-zero on any out-of-tolerance value.

---

4. Critical Dataset Rule

`Transaction_ID` IS NOT ASSUMED TO BE UNIQUE

Although the problem statement describes `Transaction_ID` as a unique transaction identifier, **the supplied CSV contains repeated `Transaction_ID` values** — approximately 2,252 rows' worth of surplus **[DATA-PRELIM]**.

4.1 Prohibitions

	Because
Use `Transaction_ID` as the database primary key	It is not one. The insert will either fail or silently collapse two real events
Delete duplicate `Transaction_ID` rows	Deleting a row destroys evidence that may be the one a court asks about
Overwrite one transaction with another	An upsert keyed on the ID loses a real transfer
Assume repeated IDs are automatically errors	Two different transfers may legitimately carry the same source-system identifier. We do not know the cause — an open question for the mentor (§30)
Silently "fix" the data in any way	Any correction must be recorded, reversible and visible in the profile

4.2 The replacement: `source_row_id`

Create an internal identifier that is guaranteed unique and immutable:

```
source_row_id  =  the 0-based ordinal of the row in the original CSV
                  (header excluded), assigned at read time,
                  never reassigned, never reused

source_row_id = 184392
Transaction_ID = TXN123456
```

The original `Transaction_ID` is **still preserved** as an indexed, non-unique attribute.

If multiple rows share the same `Transaction_ID`:

```
source_row_id 184392  ->  TXN123456     amount 25,000   22 Sep 11:02:41
source_row_id 184393  ->  TXN123456     amount  7,400   24 Sep 16:41:08
```

Both rows must remain independently traceable — in the fact store, on the graph edge, in the evidence object, in the API response, and in the citation inside a generated document. **MUST**

4.3 Duplicate classes

Not all repeats are the same thing, and the system must distinguish them:

Class	Definition	Handling
`UNIQUE`	The ID appears once	Normal
`SAME_ID_DIFFERENT_ATTRIBUTES`	Same ID, different timestamp / amount / parties — **two real events**	Both preserved, both citable, citations **must** disambiguate by date-time + amount + counterparty

Class	Definition	Handling
`EXACT_DUPLICATE`	Every field identical	Both rows preserved; counted **once** in value aggregates (`DEDUPE_EXACT_IN_AGGREGATES = True`), and the decision is recorded in the profile

4.4 Row surplus vs rows affected — do not conflate these

```
row surplus     = total_rows - distinct_ids          = 2,252   [DATA-PRELIM]
rows affected   = sum of rows belonging to repeated IDs
                = between 2,253 and 4,504, depending on the repeat distribution
```

If every repeated ID appears exactly twice, 2,252 IDs are involved and **4,504 rows** are affected. The profiler emits the repeat histogram `{repeat_count -> n_ids, n_rows}` and settles it. Any UI or document that quotes a duplicate figure must say **which** figure it is quoting.

4.5 Consequences that propagate

Layer	Consequence
Schema	`source_row_id` is the primary key; `Transaction_ID` is a non-unique index
Graph	Every edge carries the list of `source_row_id`s behind it
Detection	Aggregations run over rows, with exact duplicates collapsed where value is summed
API	`GET /api/transactions/{id}` **returns a list**; `GET /api/rows/{source_row_id}` returns one row
Evidence	Every fact carries `source_row_id`; validator **V11** rejects an ambiguous citation
Documents	A duplicated ID is cited as `TXN123456 (row 184392, 22-09-2026 11:02:41, ₹25,000)`
UI	The row id is visible in the transaction detail; an ambiguous pin opens a row picker

---

5. Source Schema

The dataset contains **11 columns**:

```
 1  Transaction_ID        7  Timestamp
 2  Sender_Account        8  Payment_Mode
 3  Receiver_Account      9  Narration
 4  Sender_IFSC          10  IP_Address
 5  Receiver_IFSC        11  Device_Type
 6  Amount
```

5.1 Header handling

Column names are resolved through an **alias map** before anything else runs, so a renamed header in a re-issued file is a five-minute fix rather than a rewrite: **[DEC]**

```
ALIASES = {
    "transaction_id": ["transaction_id", "txn_id", "transactionid", "id"],
    "sender_account": ["sender_account", "from_account", "payer_account", "debit_account"],
    "receiver_account": ["receiver_account", "to_account", "payee_account", "credit_account"],
    "sender_ifsc": ["sender_ifsc", "from_ifsc", "payer_ifsc"],
    "receiver_ifsc": ["receiver_ifsc", "to_ifsc", "payee_ifsc"],
    "amount": ["amount", "txn_amount", "value"],
    "timestamp": ["timestamp", "txn_time", "datetime", "date_time"],
    "payment_mode": ["payment_mode", "mode", "channel"],
    "narration": ["narration", "remarks", "description", "particulars"],
    "ip_address": ["ip_address", "ip", "client_ip"],
    "device_type": ["device_type", "device", "channel_device"],
}
```

Matching is case-insensitive and ignores spaces and underscores. **An unresolvable column fails the ingest job loudly, naming the offending headers — it never proceeds with a guessed mapping. MUST**

5.2 Column count is asserted, not assumed

`column_count == 11` is part of `profile-check`. A 9- or 13-column file is a different dataset and must be treated as one.

6. Column Definitions

Each column below gives: type, meaning, normalization rules, a worked example, the failure mode it guards against, and what is derived from it.

6.1 `Transaction_ID`

Type `string` · **Meaning** source-provided transaction identifier

Rule	Detail
Preserve the original value	Stored verbatim, after trimming only
Trim whitespace	Leading/trailing only — never internal
Normalize case	Uppercase **only if** profiling shows mixed case for otherwise-identical IDs; otherwise leave untouched **[DEC]**
DO NOT assume uniqueness	§4. Indexed, not unique
Provenance	Use `source_row_id` for guaranteed row-level identity

```
raw     "  TXN0000123456 "
stored  "TXN0000123456"        indexed, non-unique
key     source_row_id = 123456  unique, immutable
```

Guards against: a primary-key violation on load, or worse, a silent upsert that overwrites one real transfer with another.

6.2 `Sender_Account`

Type `string` · **Expected** 12-digit bank account identifier

Rule	Detail
Remove non-digit characters	Spaces, hyphens, `XXXX` masks if present
Zero-pad to 12 digits	After digit extraction, if shorter
NEVER convert to integer	**MUST**
Leading zeros preserved	They are part of the identifier
Validate	`^[0-9]{12}$`; a non-conforming value is flagged in the profile, **not dropped**

```
"001234567890"   must remain   "001234567890"
NOT 1234567890   NOT 1.23456789e9
```

Guards against: the most insidious bug available here. `int("001234567890") → 1234567890` creates a different account that may also exist in the dataset, merging two parties into one node. In a JSON response, any value beyond 2^53 additionally loses precision. Account numbers are identifiers, not quantities — they stay strings end to end: CSV → DuckDB → graph dictionary → API → UI → PDF.

Derived: `src_idx` (dense integer index, §11), `sender_bank` (from IFSC).

6.3 `Receiver_Account`

Type `string` · **Expected** 12-digit account identifier · **Rules** identical to 6.2.

Derived: `dst_idx`, `receiver_bank`.

> Sender and receiver are normalized by the **same function**, and both feed the **same** account dictionary. An account that appears only as a receiver is still a first-class node with a full feature row (§19).

6.4 `Sender_IFSC`

Type `string` · **Expected** Indian IFSC code

Rule	Detail
Uppercase	`sbin0001234` → `SBIN0001234`
Trim whitespace	
Validate format	`^[A-Z]{4}0[A-Z0-9]{6}$` — 11 characters, 5th is always `0`
Extract bank code	First four characters
Invalid values	Counted in `profile.malformed_ifsc`, kept verbatim, **never dropped**

```
SBIN0001234
^^^^           bank code  = SBIN
    ^          reserved   = 0
     ^^^^^^    branch code
```

Derived: `sender_bank` (4 chars). Used for the "unique sender banks" feature (a mule network spanning many banks is structurally different from one inside a single bank), and for addressing the freeze requisition to the right institution.

6.5 `Receiver_IFSC`

Same normalization rules as 6.4. **Derived:** `receiver_bank`.

6.6 `Amount`

Type `float64` for analytics, `Decimal` **for evidence and documents** · **Meaning** transaction value in INR

Rule	Detail
Must be positive	`amount <= 0` is counted in the profile and flagged; rows are retained
Retain monetary precision	Two decimal places
Avoid floating-point error in legal calculations	Sum with `Decimal`, or in integer paise, wherever a number reaches a document
Display	`₹45,000.00` — Indian grouping, formatted at the presentation layer only

```
analytics      float64           fast, vectorised, good enough for scoring
evidence       Decimal / paise   exact, auditable, what the PDF prints

0.1 + 0.2 == 0.30000000000000004       <- never in a freeze requisition
```

[DEC] The fact store holds `amount` as `DECIMAL(18,2)` in DuckDB and `float64` in the NumPy/feature path. The EvidenceObject converts to `Decimal` before any total is computed for a document, and totals printed in documents are recomputed from source rows rather than carried forward from the analytics path.

Observed distribution (§3.2) is the calibration basis for every rupee threshold in the system.

6.7 `Timestamp`

Type `datetime` · **Meaning** the time of the transaction

Required derived representation	Type	Purpose
`timestamp`	`datetime` (naive local)	Display, evidence, document citation
`ts_epoch`	`int32` / `int64` seconds	Graph arrays, temporal comparisons in compiled code
`day_index`	`uint8` 0–14	Day bucketing, "day 3 of 15"
`minute_bucket`	`int32`	Minute-level timeline filtering (§7 of the UI spec)

```
"2026-09-22 11:02:41"
     -> timestamp     2026-09-22T11:02:41
     -> ts_epoch      1790074961
     -> day_index     7
     -> minute_bucket 29834582
```

Rule	Detail
Parse strictly	A format that does not parse fails the ingest job rather than becoming `NaT`
No timezone invention	Treated as naive local time **[ASSUME]** — the dataset carries no zone. Recorded as an assumption; all comparisons are internally consistent
Window	~15 days, 2026-09-15 → 2026-09-29 **[DATA-PRELIM]**; the real bounds come from the profiler
Ordering	`ts_epoch` ascending is the canonical order for FIFO attribution and timeline playback

Temporal causality is not optional: an outgoing transfer can only carry funds that arrived before it. Every traversal and pass-through calculation depends on this column being correct.

6.8 `Payment_Mode`

Expected values `UPI` · `IMPS` · `NEFT` · `RTGS` · normalize to uppercase · unknown → `OTHER`

Rule	Detail
Uppercase and trim	
Enum discovered, not assumed	The profiler emits the observed set; a new value is a finding, not a crash
Unknown bucket	`OTHER`, counted in the profile

A forensic note worth recording: profiling observed ~59,823 `RTGS` rows in a dataset whose median transaction is ~₹899 **[DATA-PRELIM]**. Real RTGS has a ₹2 lakh floor. The profiler therefore emits an **amount × payment-mode cross-tab**, and the system **MUST NOT** attach real-world semantics to the mode field (such as "RTGS implies a large transfer"). **[DEC]** Treat `Payment_Mode` as a categorical label in synthetic data, useful for description and filtering — not as a source of amount expectations.

6.9 `Narration`

Type `string` · **Meaning** transaction remarks / transfer description

The narration may contain useful behavioural indicators (§25).

Rule	Detail
Preserve verbatim	Stored unmodified; keyword matching happens on a lowercased copy
Case-insensitive matching	
Never reinterpreted for display	The UI prints what the bank wrote, in monospace

> **IMPORTANT: narration is evidence/signalling information. It is NOT automatically proof of criminal activity.** [PS-adjacent]
>
> `WALLET_TOPUP` describes millions of entirely ordinary payments. Its contribution is as one weighted input among many, always combined with graph position and money-flow behaviour.

Derived: `narr_crypto`, `narr_wallet`, `narr_cashout`.

6.10 `IP_Address`

Type `string` · **Meaning** originating client IP address

The dataset may contain normal residential IPs and suspicious foreign/proxy-like patterns.

Derived	Definition
`ip_octet1`	First octet as an integer
`is_foreign_ip`	`ip_octet1 ∈ FOREIGN_IP_PREFIXES` — **configurable**

The problem statement specifically identifies **`185.x.x.x`** and **`194.x.x.x`** as suspicious patterns [PS]. In `config.py`:

```
FOREIGN_IP_PREFIXES = (185, 194)      # [PS] — configurable, never inlined
```

Rule	Detail
Configurable indicator	Never a hard-coded literal in detection code
Not proof	A foreign IP is a weighted signal. VPNs, travel and CDN egress all produce them
Privacy in display	Partially masked by default in the UI; full value available on the evidence record

---

6.11 `Device_Type`

Expected examples `Android` · `iOS` · `Windows_Browser` · `Web_Emulator` · `Linux_Script`

Derived	Definition
`is_automation_device`	`Device_Type ∈ AUTOMATION_DEVICES` — **configurable**

```
AUTOMATION_DEVICES = ("Web_Emulator", "Linux_Script")    # configurable
```

> **A suspicious device indicator increases evidence/risk. It does NOT automatically mean the account is fraudulent.** [PS-adjacent]
>
> The forbidden rule, stated so it can be grepped for and deleted if it ever appears:
>
> `text if device_type == "Linux_Script": account.is_mule = True # FORBIDDEN`

The enum is **discovered** by the profiler, not assumed. A sixth device value in a re-issued file must widen the distribution table, not break ingestion.

---

7. Observed Payment Distribution

Initial dataset profiling **[DATA-PRELIM]**:

```
    UPI       ~ 1,299,844      65.0%
    IMPS      ~   441,199      22.1%
    NEFT      ~   199,134      10.0%
    RTGS      ~    59,823       3.0%
                ---------
    total       2,000,000     100.0%     <- sums exactly; a good consistency sign
```

The ingestion profiler must calculate the actual final distribution. These figures are published by `GET /api/ingest/profile` from measurement, never from this document.

8. Observed Device Distribution

Initial profiling **[DATA-PRELIM]**:

```
Windows_Browser   ~ 666,004      33.30%
iOS               ~ 665,917      33.30%
Android           ~ 665,425      33.27%
Web_Emulator      ~   1,327       0.066%
Linux_Script      ~   1,327       0.066%
                    ---------
total              2,000,000
```

Two observations that shape the design:

1. The three "normal" devices are near-perfectly balanced at ~33.3% each — a strong indication of synthetic generation. Do not read behavioural meaning into the normal device mix.
2. The two automation devices appear **exactly 1,327 times each**. That precision is the first clue to the co-occurrence finding in §9.

These values must not be hard-coded.

---

9. Suspicious Dataset Signals

9.1 The signals present in the supplied dataset

Signal	Definition	Observed rows
Suspicious IP	`185.x.x.x`, `194.x.x.x`	~2,654
Automation device	`Web_Emulator`, `Linux_Script`	~2,654 (1,327 + 1,327)
Crypto narration	`P2P_CRYPTO` and related keywords	~1,327
Wallet narration	`WALLET` and related keywords	~1,327

These signals exist in the supplied dataset and **must be converted into derived features** (§10, §24).

9.2 The co-occurrence finding — read this before weighting anything

```
foreign-IP rows                ~ 2,654
automation-device rows         ~ 2,654       (1,327 + 1,327)
crypto + wallet narration rows ~ 2,654       (1,327 + 1,327)
```

Three independently derived counts landing on the same number is not a coincidence. The likely construction is **one injected set of ~2,654 transactions, each carrying a foreign IP and an automation device and one of {crypto, wallet} narration**. **[ASSUME]**

Why this matters enormously

If signals are independent	If signals co-occur (likely here)
Four signals on one account = four corroborating facts	Four signals on one account = **one** fact, counted four times
Blending them is sound	Blending them quadruple-counts a single injected flag and can push an account to CRITICAL on one piece of evidence

Required handling [DEC]:

1. The profiler emits a `signal_cooccurrence` **cross-tab** and an observed co-occurrence rate.
2. The risk model's terminal component is a **composite with a co-occurrence discount** driven by that measured rate — at a rate of 1.0 it collapses to a single-signal score.
3. A hard `TERMINAL_SIGNAL_SOLO_CAP` prevents terminal evidence alone from producing a critical score.
4. **This is the one open question the team can answer itself, in Phase 1, without a mentor.** Run the profiler; read the cross-tab.

9.3 Signal sparsity — what these signals cannot do

```
2,654 rows  =  0.1327% of all transactions
            ->  at most ~5,308 account-endpoints touched
            ->  against a PS-described 1,500 mule accounts,
                they can plausibly mark only the L3 subset
```

Layer	Terminal signal expected?	Found by
L1 Collector	**No**	Topology + velocity

Layer	Terminal signal expected?	Found by
L2 Distributor	**No**	Fan-out + splitting + pass-through
L3 Terminal	Yes	Signals add **precision**, not recall

Consequence for effort allocation: enriching the keyword list cannot improve L1/L2 detection, because those accounts carry no signals at all. Recall comes from topology and behaviour. Spend the hours there.

---

10. Derived Transaction Features

The eleven source columns are never enough on their own. During ingestion the system computes a fixed set of **derived transaction features** — deterministic functions of the source row, computed once, stored, and never recomputed differently anywhere else.

Derived field	Type	Source	Definition
`source_row_id`	`int64`	row position	0-based ordinal of the row in the source file. **Immutable, unique, the real primary key**
`src_idx`	`int32`	`Sender_Account`	Dense account index (§11)
`dst_idx`	`int32`	`Receiver_Account`	Dense account index (§11)
`ts_epoch`	`int64`	`Timestamp`	Seconds since epoch
`day_index`	`uint8`	`Timestamp`	0-based day within the dataset window
`minute_bucket`	`int32`	`Timestamp`	Minute-resolution bucket for timeline filtering
`amount_f`	`float64`	`Amount`	Analytics copy
`amount_d`	`DECIMAL(18,2)`	`Amount`	Evidence copy
`sender_bank`	`char(4)`	`Sender_IFSC`	First four characters
`receiver_bank`	`char(4)`	`Receiver_IFSC`	First four characters
`mode_code`	`uint8`	`Payment_Mode`	Categorical code; `OTHER` for unknown
`device_code`	`uint8`	`Device_Type`	Categorical code
`ip_octet1`	`uint8`	`IP_Address`	First octet
`is_foreign_ip`	`bool`	`ip_octet1`	`∈ FOREIGN_IP_PREFIXES`
`is_automation_device`	`bool`	`Device_Type`	`∈ AUTOMATION_DEVICES`
`narr_crypto`	`bool`	`Narration`	Crypto keyword hit (§25)
`narr_wallet`	`bool`	`Narration`	Wallet keyword hit (§25)
`narr_cashout`	`bool`	`Narration`	Cash-out keyword hit (§25)
`terminal_signal_count`	`uint8`	the five above	Number of distinct signal **families** present, 0–4
`dup_txn_id`	`bool`	`Transaction_ID`	True if this ID appears on more than one row (§4)

Rules for this table [DEC]:

1. Derived features are computed **once**, during ingestion, by a single function per feature.
2. No API handler, no scorer, and no UI component recomputes any of them with its own logic.
3. Every boolean above is driven by a value in `config.py` — never an inline literal.
4. `terminal_signal_count` counts **families** (ip / device / narration), not raw booleans, which is the first structural defence against the co-occurrence problem in §9.2.

```
source row  ->  normalize  ->  derive  ->  store (DuckDB + arrays)
                                 |
                                 +-- deterministic
                                 +-- config-driven
                                 +-- computed exactly once
```

11. Account Encoding and the Dense Integer Index

24,873 accounts **[DATA-PRELIM]**, identified by 12-character strings, participating in 2,000,000 edges. String comparison at that scale is the difference between a 400 ms trace and a 40 s trace.

The encoding [DEC]:

```
account_id (string)        idx (int32)
"551100220033"      <->      0
"402938475610"      <->      1
"918273645501"      <->      2
...                          ...
                           24,872

id_to_idx : dict[str, int32]     built once at ingest
idx_to_id : list[str]            exact inverse, order-stable
```

Rule	Detail
Built once	At ingest, from the union of sender and receiver accounts
Order	Assigned in first-appearance order over the file, making it reproducible for a given file
Strings never leave the boundary	Integers are internal. Every API response carries the **string** account ID
Perfect round-trip	`idx_to_id[id_to_idx[a]] == a` for every account — asserted in tests
Not persisted as meaning	An index is a run-local handle, valid only for one `dataset_fingerprint` (§28)

Why both directions matter: the UI asks about `"402938475610"`; the CSR traversal walks `idx 1`; the evidence record must print `"402938475610"` again. A failure in either direction is a silent identity error, which is why it is asserted rather than assumed.

```
API in    "402938475610"   -> id_to_idx -> 1
compute                       CSR walk over int32
API out   1 -> idx_to_id  -> "402938475610"
```

12. Graph Model: a Directed, Weighted Multigraph

12.1 The model

```
node    = bank account              (24,873)
edge    = one transaction           (2,000,000)
direction = sender -> receiver
weight    = amount
time      = ts_epoch
```

12.2 Parallel edges are real and must be preserved

Multiple transactions between the same pair of accounts are NORMAL and must be preserved as separate edges — never collapsed into one.

Profiling observed **1,996,771 unique sender→receiver pairs** across 2,000,000 transactions **[DATA-PRELIM]** — so parallel edges exist, and they are exactly the behaviour an investigator cares about.

```
402938475610 -> 918273645501   Rs  49,000   10:02:11
402938475610 -> 918273645501   Rs  49,500   10:04:52
402938475610 -> 918273645501   Rs  48,750   10:07:30

collapsed to one edge of Rs 147,250 :
    - the structuring pattern disappears
    - the 5-minute velocity disappears
    - three source_row_ids become one unciteable aggregate
```

Rule	Detail
MUST NOT de-duplicate transactions between the same pair	
Each edge carries its own `source_row_id`	So any edge in any rendered graph is traceable to one source row
Aggregation is a **view**, never a storage decision	The UI may display a bundled edge labelled "3 transfers · Rs 147,250", expanding to the three underlying rows
Self-loops	`sender == receiver` rows are retained and flagged, never silently dropped

12.3 Storage

Adjacency is held as **CSR (compressed sparse row) integer arrays** in both directions — forward for outbound traversal, reverse for inbound — at roughly 128 MB per direction for this dataset **[DATA-PRELIM]**. NetworkX is **not** in the query hot path.

---

13. Directionality: Every Account Both Sends and Receives

An account can be BOTH a sender and a receiver. This is not an edge case — it is the normal case, and the entire mule pipeline depends on it.

```
                incoming                          outgoing
victims  ---------------->  [ ACCOUNT X ]  ---------------->  next layer
(fan-in)                                          (fan-out)
```

Fact	Consequence for implementation
The same account ID appears in `Sender_Account` and in `Receiver_Account`	One account dictionary, built from the **union** of both columns
In-degree and out-degree are independent measures	Both are computed and stored for every account
Receiving is not passive	A pure receiver is a **holder**; a receiver that re-sends is a **conduit**

Therefore the system maintains, for every single account:

```
in_degree          distinct senders to this account
out_degree         distinct receivers from this account
in_txn_count       number of incoming transactions
out_txn_count      number of outgoing transactions
total_in_amount    sum of incoming value
total_out_amount   sum of outgoing value
net_balance        total_in_amount - total_out_amount   (observed flow, NOT a bank balance)
first_seen_ts      earliest transaction touching the account
last_seen_ts       latest transaction touching the account
```

> **`net_balance` is a flow residual within the dataset window, not the account's bank balance.** It says "of what we observed arriving, this much did not observably leave." It must be labelled that way everywhere it is displayed, and it must never appear in a document as an assertion about available funds.

---

14. Fan-In and Fan-Out

14.1 Definitions

Term	Definition	Signal
Fan-in	Many distinct senders → one receiver	Collection behaviour
Fan-out	One sender → many distinct receivers	Distribution / layering behaviour

```
FAN-IN                              FAN-OUT

V1 --\                                    /--> R1
V2 ----\                                 /----> R2
V3 ------>  [ COLLECTOR ]     [ DISTRIBUTOR ] ----> R3
V4 ----/                                 \----> R4
V5 --/                                    \--> R5

in_degree = 5                       out_degree = 5
```

14.2 The measurement rules [DEC]

1. Degree is counted over **distinct counterparties**, not transactions. Twenty transfers from one sender is `in_degree = 1`, `in_txn_count = 20` — a different behaviour from twenty different senders, and the two must never be conflated.
2. Degree is **always** evaluated against the dataset's own distribution, never against a number someone remembered from a different case. Mean degree here is ~80 in and ~80 out **[DATA-PRELIM]** — a threshold of "in_degree > 10 is suspicious" would flag most of the dataset.
3. Thresholds are **percentile-based and configurable**, resolved from the measured distribution at ingest, and reported with the percentile in the reason string.
4. A high degree alone is **not** a finding. Merchant accounts, payroll accounts and aggregators all show extreme degree legitimately. Degree becomes meaningful when combined with pass-through behaviour, velocity and victim-relative position.

14.3 What they are not

```
high in_degree    != collector
high out_degree   != distributor
high both         != hub        (it may just be a merchant)
```

The hub guard (§20.4) exists precisely because some high-degree accounts are infrastructure, not participants.

15. Collector Is Not "Only Receives"

A Collector account is NOT just "an account that only receives".

```
A COLLECTOR TYPICALLY:
  - receives from MANY victims                (fan-in)
  - receives in a SHORT time window           (velocity)
  - then FORWARDS funds onward                (pass-through)

It still SENDS money. That is the entire point of the role.
```

Wrong model	Correct model
`out_degree == 0` → collector	Collector = **high fan-in + short collection window + onward forwarding**
Collector is a terminal state	Collector is a **position in a flow**
Collecting excludes sending	A collector that never sends is a **holder**, which is a different finding

Why the wrong model is dangerous here: a filter of `out_degree == 0` would discard the L1 collectors in the middle of an active pipeline — the single most important accounts to freeze — and retain only dormant endpoints. It inverts the result.

Implementation consequence: the collector score is a **composite** over fan-in, collection-window compression, and forward ratio. There is no branch anywhere in the codebase of the form `if out_degree == 0: role = "COLLECTOR"`.

---

16. Distributor Is Not "Only Sends"

A Distributor account is NOT just "an account that only sends".

```
A DISTRIBUTOR TYPICALLY:
  - RECEIVES a large consolidated amount       (from the collector layer)
  - SPLITS it into many smaller outgoing transfers
  - sends to MANY different accounts           (fan-out)
```

Wrong model	Correct model
`in_degree == 0` → distributor	Distributor = **receives consolidated value, then splits it outward**
A distributor has no inbound	A distributor is defined by its inbound: it must receive before it can split

The dual-sided truth, stated once for both roles:

> **Collector and Distributor are role descriptors of behaviour, not exclusive capabilities. Both roles send. Both roles receive. The difference is the shape of the flow through the account, not the presence or absence of a direction.**

Required UX consequence (binding on the frontend spec): every account panel shows inbound **and** outbound behaviour side by side, for every role, with no role hiding one side of the ledger.

```
ACCOUNT 402938475610        role: COLLECTOR (assessed)

INBOUND                              OUTBOUND
in_degree        47                  out_degree        3
in_txn_count    312                  out_txn_count    12
total_in        Rs 24,10,500         total_out        Rs 23,86,000
window        2.4 hours              forward ratio    0.99
```

17. The Layer Model (L1 / L2 / L3)

Mule networks in this domain are commonly described in three layers. The system uses these terms **as assessed descriptors with reasons**, never as asserted facts.

```
VICTIM
  |
  |  (1 hop)
  v
L1  COLLECTOR / "money mule"
    receives victim funds directly
    high fan-in, short window, forwards quickly
  |
  |  (2 hops)
  v
L2  DISTRIBUTOR / LAYERING
    receives consolidated funds, splits and disperses
    high fan-out, pass-through, structuring
  |
  |  (3+ hops)
  v
L3  TERMINAL / EXIT
    crypto, wallet, cash-out, foreign IP, automation device
    flow stops being observable here
```

Layer	Typical behaviour	Typical evidence	Terminal signals?
L1	Receives from many victims, forwards fast	Fan-in, velocity, pass-through	No
L2	Receives consolidated, splits outward	Fan-out, structuring, pass-through	No
L3	Converts or exits the banking system	Narration keywords, foreign IP, automation device, low forward ratio	Yes

Classification rules [DEC] — all four are mandatory:

1. **Do NOT classify a layer from hop depth alone.** A victim's own bank's sweep account is one hop away and is not an L1 collector.
2. **Do NOT classify from in-degree alone.** High fan-in with no forwarding is a holder or a merchant.
3. **Do NOT classify from out-degree alone.** High fan-out with no matching inbound consolidation is a payroll or disbursement account.
4. A layer assignment is emitted **only** with a composite reason string naming the measurements that produced it, and is always rendered as **ASSESSED**, never as OBSERVED.

```
layer = f(hop_distance_from_victim,
          inbound_shape,
          outbound_shape,
          pass_through_behaviour,
          temporal_compression,
          terminal_signals)
```

18. Layering Is Victim-Relative, Not Global

There is no global "L1 account" in this dataset. L1 only means "one hop from this victim, in this case".

```
CASE A   victim 551100220033
         L1 = 402938475610
         L2 = 918273645501

CASE B   victim 667788990011
         L1 = 918273645501        <- the same account, now L1
         L2 = 774411220099
```

Rule	Detail
Layer is a **per-investigation** property	It is computed during a trace, relative to the trace's seed victim
It is **not** an account attribute	It is **never** stored on the account record as a global label
An account may hold different layers in different cases simultaneously	And both are correct
The API reflects this	Layer appears in **trace** responses; the account profile endpoint reports topology and behaviour, not a global layer

Why this is a correctness issue, not a modelling preference: a globally stored layer would let one investigation's conclusion leak into another investigator's case file as if it were an established fact about the account. That is contamination of evidence, and it would be visible to any reviewer who compared the two files.

Implementation consequence: `layer` is a field on a trace-node object, carrying `relative_to_victim` and the hop distance that produced it.

```
trace_node {
  account_id          "918273645501",
  layer               "L2",
  relative_to_victim  "551100220033",
  hop_distance        2,
  layer_reason        "2 hops from victim; receives consolidated inflow from 1 L1
                       collector; splits to 23 distinct receivers within 51 minutes"
}
```

19. The Account Feature Row

Every account in the dataset — all ~24,873 of them, whether it ever appears in an investigation or not — has exactly one feature row, computed at ingest.

Group	Fields
Identity	`account_id`, `idx`, `banks_used[]`, `first_seen_ts`, `last_seen_ts`, `active_days`
Topology	`in_degree`, `out_degree`, `in_txn_count`, `out_txn_count`, `distinct_counterparties`
Value	`total_in_amount`, `total_out_amount`, `net_balance`, `avg_in_amount`, `avg_out_amount`, `max_in_amount`, `max_out_amount`
Temporal	`peak_in_window_count`, `peak_out_window_count`, `median_hold_seconds`, `collection_window_seconds`
Flow	`forward_ratio`, `pass_through_score`, `split_ratio`
Signals	`foreign_ip_txn_count`, `automation_txn_count`, `crypto_narr_count`, `wallet_narr_count`, `cashout_narr_count`, `distinct_signal_families`
Context	`mode_mix{}`, `device_mix{}`, `unique_sender_banks`, `unique_receiver_banks`

Rules [DEC]:

1. The feature row contains **measurements only**. No score, no role, no layer, no verdict.
2. Scores are computed **on top of** the feature row, at query time, by the risk engine — so a threshold change in `config.py` changes scores without re-ingesting 2M rows.
3. Every number in the feature row must be reproducible by a SQL query over the fact table. If it cannot be, it does not belong here.
4. Percentile ranks are attached at query time from the measured distribution, so a reason string can say "in_degree 47 is at the 99.4th percentile for this dataset" instead of the meaningless "in_degree 47 is high".

---

20. Pass-Through Behaviour

20.1 What it is

Pass-through is the single most informative behaviour in this dataset. A mule account is a pipe, not a wallet: value arrives and leaves quickly, and most of what arrives leaves.

```
NORMAL ACCOUNT                    PASS-THROUGH ACCOUNT

in   Rs 85,000  over 30 days      in   Rs 24,10,500  over 2.4 hours
out  Rs 61,000  over 30 days      out  Rs 23,86,000  over 3.1 hours
hold days to weeks                hold median 4 minutes
forward ratio  0.72, diffuse      forward ratio  0.99, immediate
```

20.2 The three measurements

Measure	Definition	Reads as
`forward_ratio`	`total_out_amount / total_in_amount`, clipped to `[0, 2]`	How much of what arrived left again
`median_hold_seconds`	Median of (outgoing timestamp − matched incoming timestamp), FIFO-attributed	How long value rested
`pass_through_score`	Composite of the two, scaled against the dataset distribution	Pipe-likeness

FIFO attribution rule [DEC]: outgoing value is matched against the earliest unmatched incoming value, respecting **temporal causality** — an outgoing transfer may only be matched to inflows with a **strictly earlier** `ts_epoch`. Value that cannot be matched to any prior inflow is reported as `unattributed_out_amount` rather than being forced into a match.

```
in   10:02  Rs 50,000  ---\
in   10:05  Rs 40,000  ----\
                            >--  out 10:09  Rs 85,000
                           /         matched: 50,000 (7 min) + 35,000 (4 min)
in   11:40  Rs 20,000  ---X         NOT matched: later than the outflow
```

20.3 Rules

1. A high `forward_ratio` with a **long** hold time is not pass-through — it is ordinary account turnover.
2. A short hold time with a **low** forward ratio is not pass-through — it is a partial sweep.
3. Pass-through requires **both**, and the reason string must quote both numbers.
4. `forward_ratio > 1` is possible and legitimate — the account had prior balance, or inflows outside the window. It is reported, never clipped silently to 1.0, and never treated as an error.

20.4 The hub guard

Some accounts are **infrastructure**: settlement accounts, aggregators, large merchants. They exhibit extreme degree, high throughput and fast forwarding, and they will dominate every traversal if left unguarded.

Guard	Behaviour
Hub detection	An account whose degree exceeds `HUB_DEGREE_PERCENTILE` (configurable) of the measured distribution is marked `is_hub`

Guard	Behaviour
Traversal	Expansion **through** a hub is suppressed; the hub is still returned as a node, with `truncation_reason: "HUB_GUARD"`
Transparency	The UI states that expansion stopped at a hub, with the hub's degree — it never silently prunes
Not an accusation	`is_hub` is a structural property, not a risk signal, and contributes **nothing** to the risk score

---

21. Velocity Windows

21.1 Why velocity matters

Money laundering is a **race**: funds must move before the victim notices, before the bank flags it, and before a freeze can land. Compression in time is therefore one of the few behaviours that is hard for an adversary to avoid.

```
15-day dataset window
|------------------------------------------------------------|
                   ^^^^
                   47 transfers into one account in 2.4 hours
```

21.2 The window set [DEC]

Velocity is evaluated over a **fixed, configurable set of rolling windows**, so that bursts at different timescales are all visible:

```
VELOCITY_WINDOWS = (
    300,      #  5 minutes   burst / structuring
    3600,     #  1 hour      collection sprint
    21600,    #  6 hours     working session
    86400,    # 24 hours     daily pattern
)
```

For each account and each window the system stores the **peak** count and **peak** value, in both directions:

Field	Meaning
`peak_in_count_{w}`	Maximum number of incoming transactions in any rolling window of `w` seconds
`peak_in_amount_{w}`	Maximum incoming value in any such window
`peak_out_count_{w}`	Same, outbound
`peak_out_amount_{w}`	Same, outbound
`peak_in_window_start_{w}`	Timestamp of the start of that peak window — **required**, so evidence can cite the exact interval

21.3 Rules

1. Windows are **rolling**, not calendar-aligned. A burst from 23:50 to 00:10 must be detected as one burst, not split across two days.
2. Every velocity claim in evidence cites the **window, the count, the value and the start timestamp** — "47 inbound transfers totalling Rs 24,10,500 between 2026-09-22 09:14 and 11:38".
3. Velocity thresholds are **percentile-based against the measured dataset**, not absolute counts.
4. Velocity alone is not a finding. A payroll run is high-velocity and entirely legitimate.
5. With ~133,333 transactions per day across the dataset **[DATA-PRELIM]**, ordinary accounts also show bursts. The percentile is what carries the meaning.

22. Hold Time and Dispersal Latency

Measure	Definition	Why it matters
`median_hold_seconds`	Median FIFO-matched (out − in) interval	The core "pipe vs. wallet" discriminator
`min_hold_seconds`	Fastest matched forward	Exposes automated forwarding
`dispersal_latency`	Seconds from the **first** inflow of a burst to the **first** outflow after it	How fast the account reacts to receiving money
`collection_window_seconds`	Span from first to last inflow of the largest inbound burst	How compressed the collection phase was

```
hold time          what it usually indicates
-------------------------------------------------------------
< 60 s             automated forwarding, scripted
1 - 30 min         active human operator
30 min - 6 h       operator with a batching routine
> 48 h             ordinary account behaviour
```

> The table above describes **tendencies for interpretation by an investigator**, not thresholds that the system applies as rules. No band in it ever sets a verdict. **[ASSUME]**

Rule: hold time is only defined where FIFO matching succeeded. For an account with no attributable outflows the field is `null` — **never** zero. Zero means "forwarded instantly", `null` means "we could not measure it", and conflating the two would manufacture evidence.

23. Amount Splitting and Structuring

23.1 The pattern

Large consolidated value is broken into many smaller transfers to stay under reporting or monitoring thresholds, and to spread exposure across accounts.

```
IN     Rs 9,85,000  from 1 collector              10:31
OUT    Rs   49,500  -> A    10:44
       Rs   49,000  -> B    10:46
       Rs   48,750  -> C    10:49
       Rs   49,250  -> D    10:52
       ... 19 more, all between Rs 48,000 and Rs 49,900
```

23.2 The measurements

Field	Definition
`split_ratio`	`out_txn_count / in_txn_count` over the burst
`out_amount_cv`	Coefficient of variation of outgoing amounts in the burst — **low CV with high count is the structuring signature**
`out_amount_mode_cluster`	Share of outgoing transfers falling within a narrow band of each other
`round_amount_share`	Share of transfers that are exact round numbers

23.3 Rules [DEC]

1. **No absolute rupee threshold may be hard-coded anywhere in the system.** The dataset's median transaction is **₹899.35** and its mean is **₹1,760.88 [DATA-PRELIM]** — a "₹50,000 structuring floor" borrowed from real-world AML guidance would match almost nothing here, and a "₹1,000 floor" would match over half the dataset. Every amount band is derived from the measured distribution, at ingest, and stored in the profile.
2. Structuring is assessed **relative to the account's own inbound**, not to a global constant: "23 outgoing transfers, each between 4.9% and 5.1% of the single inbound amount" is a finding; "23 transfers under ₹50,000" is not.
3. Low variance alone is not structuring. Subscription payouts and fixed-fee disbursements have very low variance.
4. The reason string quotes the count, the band, the CV and the window.

24. Terminal Signals

24.1 The five signals

Family	Signal	Source column	Config key
Network	Foreign / proxy-like IP prefix	`IP_Address`	`FOREIGN_IP_PREFIXES`
Device	Automation device	`Device_Type`	`AUTOMATION_DEVICES`
Narration	Crypto keyword	`Narration`	`CRYPTO_KEYWORDS`
Narration	Wallet keyword	`Narration`	`WALLET_KEYWORDS`
Narration	Cash-out keyword	`Narration`	`CASHOUT_KEYWORDS`

These are **three families** (network, device, narration), which is what `distinct_signal_families` counts.

24.2 The rules that govern them

1. **A terminal signal is an indicator of exit behaviour. It is NEVER proof of criminality.** It is one weighted input, always combined with graph position and money-flow behaviour.
2. The **co-occurrence discount** of §9.2 applies: because the signals appear to have been injected together, the composite must not count one injected flag three times.
3. `TERMINAL_SIGNAL_SOLO_CAP` prevents terminal evidence alone from producing a CRITICAL assessment.
4. Terminal evidence is rendered with `interpretation.verdict` permanently `null` — the system presents the observation and the possible readings, and a human decides.
5. Signals are **sparse** — ~0.1327% of rows **[DATA-PRELIM]**. They add precision at L3. They cannot find L1 or L2 (§9.3).

24.3 The shape of a terminal evidence record

```
terminal_evidence {
  account_id        "774411220099",
  observed {
    foreign_ip_txns          14,
    automation_txns          14,
    crypto_narration_txns     7,
    distinct_families         3,
    cooccurrence_rate      1.00,
    source_row_ids    [ 1842991, 1843004, ... ]
  },
  interpretation {
    possible_readings [
      "transfers originated from an automated client on a foreign-prefix IP",
      "a VPN or hosting provider used by an ordinary customer",
      "test or batch traffic from a legitimate integration"
    ],
    verdict   null                    <- permanently null, by design
  }
}
```

25. Narration Keywords

25.1 The keyword families

```
CRYPTO_KEYWORDS  = ("P2P_CRYPTO", "CRYPTO", "BTC", "USDT", "BINANCE", "WAZIRX")
WALLET_KEYWORDS  = ("WALLET", "WALLET_TOPUP", "PREPAID", "LOAD")
CASHOUT_KEYWORDS = ("ATM", "CASH", "WITHDRAW", "CASHOUT")
```

These lists live in `config.py`. They are illustrative starting points, widened only from values actually observed in the dataset by the profiler. [DEC]

25.2 Matching rules

Rule	Detail
Case-insensitive	Match on a lowercased copy; store the narration verbatim
Substring with word-ish boundaries	`"CASH"` must not fire on `"CASHBACK_REVERSAL"` — boundary-aware matching, with the exclusion list in config
Record **which** keyword matched	The evidence record names the keyword and the matched narration text
Never translated or paraphrased	The document prints the bank's own string
The profiler emits a narration frequency table	So the team can see what is actually in the file instead of guessing

25.3 The warning

> **`WALLET_TOPUP` describes millions of entirely ordinary payments. A narration keyword is a weighted signal, not an accusation.**

```
FORBIDDEN
    if "CRYPTO" in narration: account.is_mule = True

CORRECT
    crypto keyword present
      -> terminal signal family "narration" set
      -> weighted into the composite with the co-occurrence discount
      -> surfaced as OBSERVED text with its source_row_id
      -> interpreted by a human
```

25.4 Provenance

Every narration-based signal carries the `source_row_id` of the row it came from, so an investigator can open the exact transaction, see the full untruncated narration, and judge it. A keyword hit that cannot name its source row is a bug.

---

26. The Mule Risk Index (MRI)

26.1 Composition

The MRI is a 0–100 composite over six components. **The weights are the frozen v1.0 values and live in `config.py`.**

Component	Weight	Driven by
Pass-through behaviour	**25**	`forward_ratio`, `median_hold_seconds` (§20)
Velocity	**20**	Peak rolling-window counts and values (§21)
Fan-in	**15**	`in_degree` percentile, collection window (§14)
Fan-out	**15**	`out_degree` percentile, split ratio (§14, §23)
Terminal signals	**15**	Composite with co-occurrence discount (§24)
Behavioural anomaly	**10**	Amount CV, round-amount share, device/mode mix shifts
Total	**100**	

```
bands     LOW       0  - 29
          MEDIUM   30  - 59
          HIGH     60  - 79
          CRITICAL 80  - 100
```

26.2 The non-negotiable properties

Property	Requirement
Explainable	Every score ships with a per-component breakdown **and** a reason string naming the measurements and percentiles behind it
Deterministic	The same data and the same config always produce the same score. No randomness, no model drift, no LLM in the scoring path
Calibration honest	`calibration_regime: "HEURISTIC_SELF_VALIDATION"` is returned with every score, because there are no ground-truth labels (§30)
Disclaimer	A fixed `disclaimer` string accompanies every score and is displayed **verbatim**, never truncated or restyled away
Capped	No single component may drive a CRITICAL band on its own
Reversible	Changing a weight in `config.py` changes scores on the next request, with no re-ingest

26.3 What the MRI is not

> **The MRI is a prioritisation aid for a human investigator. It is not a verdict, not a probability of guilt, and not a legal finding.**

```
MRI 87 CRITICAL
  means  "this account's observed behaviour matches the mule pattern
          more strongly than 99.6% of accounts in this dataset,
          for the following six measured reasons"
  NOT    "this account is a mule"
  NOT    "87% likely to be fraud"
  NOT    "sufficient grounds for action"
```

The score is always displayed with its components. A bare number with no breakdown is a specification violation.

---

27. The Evidence Principle and Row-Level Provenance

27.1 The principle

> **Every single claim the system makes must be traceable to specific source rows.**

```
claim  ->  measurement  ->  source_row_id[]  ->  the original CSV row
```

If a number cannot name the rows it came from, it does not get displayed, it does not enter a document, and it does not enter the evidence basket.

27.2 `source_row_id` is a first-class concept

Because `Transaction_ID` is **not unique** (§4), the row ordinal is the only reliable identity in this dataset. It is therefore first-class everywhere:

Layer	Requirement
Ingest	`source_row_id` is assigned as the 0-based ordinal of the row in the source file, stored, and never changed
Fact store	Present on every fact-table row; the de-facto primary key
Graph	Every edge carries the `source_row_id` of the transaction it represents
Features	Every aggregate can be decomposed back to the set of row ids that produced it
Evidence	Every `EvidenceObject` carries `source_row_ids[]`
API	Every transaction payload includes `source_row_id`; `GET /api/rows/{source_row_id}` returns the verbatim source row
UI	Transaction and edge detail panels display `source_row_id`, copyable
Documents	Every cited transaction prints its `source_row_id` alongside its `Transaction_ID`

```
EvidenceObject {
  claim            "47 inbound transfers totalling Rs 24,10,500 in 2h24m",
  claim_type       "OBSERVED",
  measurement      { window_s: 8640, count: 47, amount: "2410500.00" },
  source_row_ids   [ 118422, 118431, 118447, ... ],      <- 47 ids
  dataset_fingerprint "sha256:9f2c...",
  generated_at     "2026-10-02T14:11:09"
}
```

27.3 The reproducibility test

> Take any number in any generated PDF. Look up its `source_row_ids`. Open those rows in the raw CSV. Recompute the number by hand. **It must match exactly.**

This test is the acceptance criterion for the evidence layer, and it is why monetary totals in documents are recomputed with `Decimal` from the cited rows (§6.6) rather than carried forward from the analytics path.

---

28. The Dataset Fingerprint

Every analysis is bound to the exact bytes it was computed from.

```
dataset_fingerprint = "sha256:" + sha256(file_bytes)[:16]
```

Rule	Detail
Computed at ingest	Over the raw source file, before any parsing
Returned in **every** API response	In `meta.dataset_fingerprint`
Printed on **every** generated document	In the header or footer block
Stored on every `EvidenceObject`	So stale evidence is detectable
Mismatch is an error, not a warning	Evidence whose fingerprint does not match the loaded dataset is rejected, with a clear message

Why it exists: six months after the event, someone will ask whether a figure in a freeze requisition came from the same file the investigation was run on. The fingerprint answers that question in one comparison. It also protects against the much more mundane failure of a re-issued dataset silently replacing the old one mid-demo.

29. What This Data Does NOT Say

This section exists to stop the system from claiming things the file cannot support.

The data does NOT contain	Therefore the system must NOT
Account-holder names, KYC, addresses, phone numbers	Produce any statement about a person; it speaks only about accounts
Bank balances	Describe `net_balance` as available funds, or state that an amount "is still in" an account
Confirmed fraud labels	Say "this is a mule account", "confirmed", "guilty" or "criminal"
Police complaints or FIR linkage	Assert that a victim reported anything; the victim is an input to a trace
Account opening dates or dormancy history	Claim an account is "newly opened" or "dormant"
Transaction reversals, chargebacks or failures	Assume every row is a settled transfer — only that it is a recorded one
Geolocation	Convert an IP prefix into a country, city or person. `185.x` is a **prefix class**, not a location **[ASSUME]**
Cross-institution context	Claim completeness: this file is a slice, and other transactions certainly exist outside it
Intent	Infer purpose from narration text, which is operator-entered and unverified

```
OBSERVED    measurable in the file, with row ids
INFERRED    derived by a documented, deterministic rule
ASSESSED    a weighted judgement, with reasons and a disclaimer
------------------------------------------------------------
ASSERTED    not available to this system, at any confidence
```

30. Ground Truth Is UNKNOWN

The dataset ships with no labels. Not one account is marked as a mule.

Consequence	Detail
No accuracy claim is possible	Precision, recall, F1 and ROC-AUC **cannot** be computed. Any such number in a UI, a document or a pitch would be fabricated
No model may be trained on labels	There are none to train on. Supervised learning is off the table for v1.0
Validation is self-consistency, not correctness	`calibration_regime: "HEURISTIC_SELF_VALIDATION"` is stated in every scored response
The PS narrative is context, not a label set	The problem statement describes ~1,500 mule accounts. That is a description of how the file was built, **not** a usable label column, and it **MUST NOT** be used to claim a hit rate
Honesty is scored	Stating this limitation plainly is a strength in judging, not a weakness. A system that claims 94% accuracy on unlabelled data is making it up

What the team may legitimately say:

```
SAY     "the system ranks accounts by a transparent, deterministic
         composite of six measured behaviours, each traceable to source rows"

SAY     "we have no ground truth, so we report calibration regime
         HEURISTIC_SELF_VALIDATION and show every component"

DO NOT SAY   "94% accurate"
DO NOT SAY   "detects 1,487 of the 1,500 mules"
DO NOT SAY   "low false positive rate"
```

What self-validation does cover: internal consistency (totals reconcile with source rows), determinism (identical input → identical output), stability (small threshold changes do not scramble the ranking), and explainability (every score decomposes).

31. Processing Rules

The fourteen rules that govern every component touching this data.

#	Rule
1	**Treat account numbers as strings.** Never integers. Never floats. Leading zeros survive end to end
2	**Never assume `Transaction_ID` is unique.** Index it, do not key on it. `source_row_id` is the key
3	**Never delete or quarantine duplicate-ID rows.** Preserve every source row with immutable provenance
4	**Preserve parallel edges.** Multiple transactions between the same pair are separate edges
5	**Respect temporal causality.** Money cannot leave before it arrives; FIFO attribution matches only strictly earlier inflows
6	**No hard-coded thresholds.** Every threshold, keyword list, prefix set and window lives in `config.py`
7	**No absolute rupee floors.** Bands are derived from the measured distribution; the median is ₹899.35 **[DATA-PRELIM]**
8	**Profile, do not assume.** Enums, distributions and bounds are discovered at ingest and published by the profiler
9	**Compute derived features once.** One function per feature, at ingest; no component recomputes them differently
10	**Use `Decimal` for money in evidence and documents.** `float64` is for analytics only
11	**Every claim carries `source_row_ids[]`.** A number that cannot cite rows is not displayed
12	**Every response carries the `dataset_fingerprint`.** Mismatches are errors
13	**Layer is per-investigation.** Never stored as a global account attribute
14	**Separate OBSERVED / INFERRED / ASSESSED.** Never assert fraud, guilt or criminality

32. The Do-Not List

Rules 1–14 restated as the failures they prevent — the list to check code against during review.

```
✘  int(account_number)                          identity destroyed
✘  PRIMARY KEY (Transaction_ID)                 ingest fails, or rows overwrite
✘  DROP DUPLICATES on Transaction_ID            evidence silently deleted
✘  GROUP BY sender, receiver  as storage        structuring pattern erased
✘  if out_degree == 0: role = COLLECTOR         inverts the result (§15)
✘  if in_degree  == 0: role = DISTRIBUTOR       inverts the result (§16)
✘  layer = hop_count                            ignores behaviour (§17)
✘  account.layer = "L1"   stored globally       cross-case contamination (§18)
✘  if "CRYPTO" in narration: is_mule = True     accusation from one word
✘  if device == "Linux_Script": is_mule = True  accusation from one field
✘  amount > 50000  as a hard-coded floor        wrong by three orders of magnitude
✘  float sums in a freeze requisition           legally embarrassing rounding
✘  "accuracy: 94%"                              fabricated; there are no labels
✘  a score with no component breakdown          unexplainable, unusable
✘  a number with no source_row_ids              uncitable, therefore not evidence
✘  NetworkX in the query hot path               seconds instead of milliseconds
✘  loading 2,000,000 rows into the browser      the UI dies
✘  an LLM anywhere in the scoring path          non-deterministic evidence
```

33. Performance Context

The data's size is a design constraint, not a footnote.

```
2,000,000 rows      ~274 MB CSV
   24,873 accounts
       15 days      ~133,333 transactions/day
                    ~128 MB CSR per direction (forward + reverse)
```

Budget	Target	Why the data makes it hard
Full ingest + profile + feature build	**≤ 60 s**	2M rows parsed, normalized, 20 derived columns, 24,873 feature rows
4-hop trace, p95	**≤ 2000 ms**	Mean degree ~80 means naive 4-hop expansion reaches the whole graph
Graph payload build, p95	**≤ 500 ms**	Capped at 800 nodes / 1500 edges
Account profile	**≤ 300 ms**	Served from precomputed feature rows

What the degree distribution forces:

```
mean out-degree ~80
  hop 1       80
  hop 2    6,400
  hop 3 512,000      <- already larger than the account universe
  hop 4      all

therefore: value-weighted traversal + hub guard + caps are
           correctness requirements, not optimisations
```

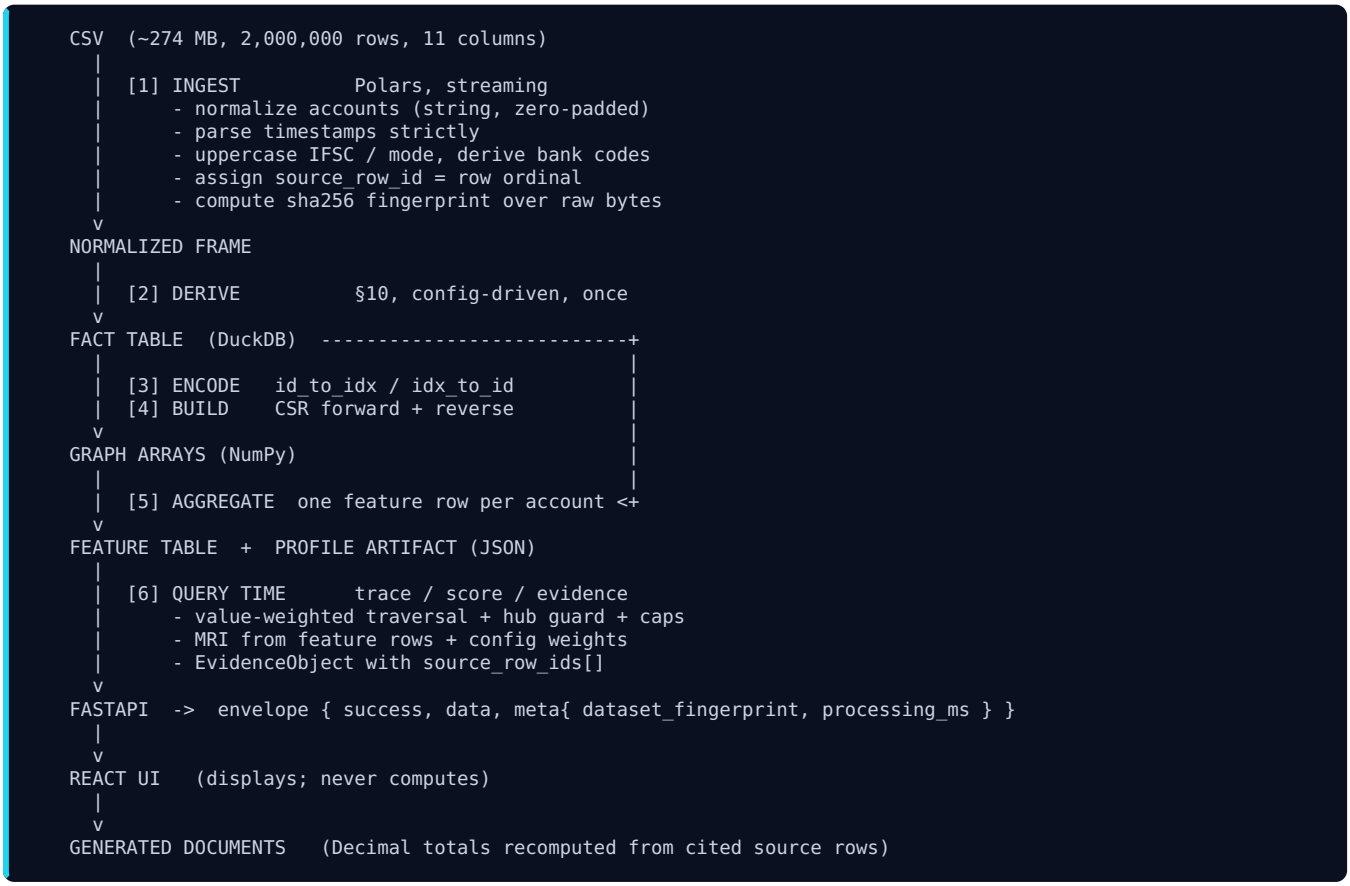
The consequences, all already in the architecture: Polars for ingest, DuckDB as the columnar fact store, NumPy CSR for traversal, precomputed feature rows, value-weighted beam expansion with a minimum-value cutoff, the hub guard, and strict payload caps with an honest `truncation_reason`.

34. Internal Stores

Store	Technology	Holds	Why
Fact table	DuckDB (columnar, on disk)	All 2M rows, source + derived columns	Fast analytical SQL, no server, exact `DECIMAL`
Graph arrays	NumPy CSR, int32	Forward and reverse adjacency, aligned `amount` / `ts_epoch` / `source_row_id` arrays	Microsecond neighbour lookups
Account map	Python dict + list	`id_to_idx`, `idx_to_id`	String↔integer boundary (§11)
Feature table	DuckDB / Arrow	One row per account (§19)	Query-time scoring without touching 2M rows
Profile artifact	JSON on disk (`data/artifacts/profile.json`)	Measured distributions, enums, bounds, cross-tabs, fingerprint	The published, inspectable truth about the file
Case store	SQLite / JSON	Cases, evidence baskets, diary entries, generated documents	Investigator state, separate from immutable dataset facts

The separation that matters: dataset facts are **immutable and shared**; case state is **mutable and per-investigation**. Nothing an investigator does can alter a measurement, and no measurement carries a previous investigator's conclusion.

35. The Data Flow



The one-way rule: data flows left to right. Nothing downstream writes back into the fact table, and nothing downstream recomputes a measurement with its own logic. The UI displays. The documents cite. The backend measures.

---

36. Example: Investigation Context

A worked end-to-end example, using the illustrative IDs reused across all ABHEDYA-CHAKRA documents.
These are illustrative values for specification purposes, not findings from the dataset. [ASSUME]

36.1 The input

```
case_id          CASE-2026-001
victim_account   551100220033
reported_amount  Rs 2,50,000
incident_window  2026-09-22 09:00  ->  2026-09-22 18:00
max_hops         4
```

36.2 What the backend does

```
[1]  resolve victim string -> idx                    id_to_idx
[2]  reverse/forward CSR expansion, value-weighted   hub guard active
[3]  per-node features pulled from the feature table
[4]  layer assigned RELATIVE TO 551100220033         §18
[5]  MRI computed per node from config weights       §26
[6]  EvidenceObjects built with source_row_ids[]     §27
[7]  payload capped at 800 nodes / 1500 edges        truncation_reason if hit
```

36.3 The shape of the result

```
trace trc_01JA8Z9V3K       fingerprint sha256:9f2c...    processing_ms 1184

VICTIM  551100220033
  |
  |  Rs 2,50,000   2026-09-22 09:14    source_row_id 118422
  v
L1  402938475610   HDFC0001234     MRI 87  CRITICAL
    in_degree 47   window 2h24m    forward_ratio 0.99   hold 4m
  |
  |  Rs 23,86,000 consolidated, 3 transfers
  v
L2  918273645501                   MRI 74  HIGH
    receives consolidated, splits to 23 receivers in 51 minutes
  |
  v
L3  774411220099                   MRI 81  CRITICAL
    crypto narration x7, foreign IP x14, automation device x14
    distinct_families 3, cooccurrence_rate 1.00, verdict null
  |
  v
HUB 999900001111                   expansion stopped
    truncation_reason HUB_GUARD, degree at 99.97th percentile
```

36.4 What the investigator sees

Element	Content
Graph	4 layers, value-weighted edges, hub marked and not expanded
Timeline	The 2h24m collection burst, then the 51-minute dispersal
Evidence basket	Each claim with its `source_row_ids[]`, exportable
Risk panel	Six components, each with its measurement and percentile
Disclaimer	Displayed verbatim with every score
Documents	Freeze requisition under **S.94 BNSS, 2023**, totals recomputed with `Decimal`

37. Example: Account Context

```
GET /api/accounts/402938475610
```

```
IDENTITY
  account_id              "402938475610"        <- string, always
  banks_used              [ "HDFC" ]
  first_seen              2026-09-22 09:14
  last_seen               2026-09-22 12:51
  active_days             1 of 15

TOPOLOGY                        percentile
  in_degree               47        99.4
  out_degree               3        41.2
  in_txn_count           312
  out_txn_count           12

VALUE
  total_in_amount         Rs 24,10,500.00
  total_out_amount        Rs 23,86,000.00
  net_balance             Rs    24,500.00    <- observed flow residual, NOT a bank balance

TEMPORAL
  collection_window       2h 24m
  median_hold             4m 11s
  peak_in_count_3600      188
  peak_in_window_start    2026-09-22 09:14

FLOW
  forward_ratio           0.99
  pass_through_score      0.94
  split_ratio             0.04

SIGNALS
  foreign_ip_txns           0
  automation_txns           0
  narration_hits            0
  distinct_families         0        <- an L1 collector carries no terminal signals (§9.3)

ASSESSMENT
  mri                      87   CRITICAL
  calibration_regime       "HEURISTIC_SELF_VALIDATION"
  role_assessment          "COLLECTOR (assessed)"
  reason                   "47 distinct senders (99.4th pct) within 2h24m;
                            99% of inflow forwarded with a median hold of 4m11s;
                            consolidated into 3 outgoing transfers"
  disclaimer               "<displayed verbatim>"

NOTE
  no layer field here       layer is per-investigation (§18)
```

Read the SIGNALS block carefully. This account scores CRITICAL with **zero** terminal signals. That is the system working correctly: recall at L1 comes from topology and behaviour, never from keywords (§9.3).

38. Rules for AI and LLM Usage

ABHEDYA-CHAKRA may use an optional local LLM (Ollama). The boundary is absolute.

The LLM	The LLM
Rephrase backend-supplied reason strings into fluent prose	Compute, estimate or adjust any number
Draft a narrative summary **from a supplied structured payload**	Decide a role, a layer, a band or a verdict
Improve the readability of a case note	Invent an account, a transaction, an amount or a timestamp
Translate fixed, pre-approved strings	Generate legal text or section citations
	Appear anywhere in the scoring, traversal or evidence path

The three mechanical rules [DEC]:

1. **Prose-only.** The LLM receives a structured payload and returns prose. Any number in its output must appear verbatim in the input payload.
2. **Anti-hallucination validation.** Generated text is scanned before display: every account ID, amount, date and count is checked against the source payload. A mismatch **blocks** the text and falls back to the deterministic template.
3. **Fully optional.** The system runs end to end with the LLM disabled, producing deterministic template prose. No feature, document or score depends on it.

```
payload (facts)  ->  LLM  ->  prose  ->  validator  ->  display
                                            |
                                            +-- mismatch -> discard, use template
```

> **The deterministic path is the product. The LLM is a presentation convenience.**

39. Source-of-Truth Hierarchy

When two sources disagree, resolve in this order. Higher wins, always.

```
1  THE SOURCE CSV                    the bytes, hashed into the fingerprint
2  THE PROFILE ARTIFACT              measured at ingest from those bytes
3  config.py                         thresholds, weights, keyword lists
4  BACKEND_API.md                    payload shapes, error codes, envelope
5  DATA_CONTEXT.md  (this document)  meaning, rules, prohibitions
6  FRONTEND_UI.md                    presentation of backend-supplied values
7  ABHEDYA_CHAKRA_PRD.md             scope, goals, plan
8  anything remembered, assumed or verbally agreed     lowest
```

Conflict	Resolution
A number in this document vs. the profiler	**The profiler wins.** Every figure here marked **[DATA-PRELIM]** is illustrative and must be recalculated
This document vs. the PRD's older data contract	**This document wins** on the `Transaction_ID` / `source_row_id` rule (§4, §27) — it supersedes the earlier assertion of uniqueness and the duplicate-quarantine behaviour
This document vs. `BACKEND_API.md`	The API doc wins on **payload shape**; this document wins on **meaning and prohibitions**
Any document vs. the data	**The data wins.** Re-profile, then correct the document

40. Developer Decision Tree

A reference for the moment a developer is about to write a line of code that touches this data.

```
Am I about to put a number on a screen or in a document?
  |
  +-- Can I name the source_row_ids behind it?
  |      NO  -> stop. It is not evidence. Do not display it. (§27)
  |      YES -> continue
  |
  +-- Did the backend compute it?
  |      NO  -> stop. The frontend does not calculate. (§39)
  |      YES -> continue
  |
  +-- Is it OBSERVED, INFERRED or ASSESSED?
  |      Label it. Never assert fraud, guilt or criminality. (§29)
  |
  +-- Is it money entering a document?
         YES -> recompute with Decimal from the cited rows. (§6.6, §27)

Am I about to hard-code a number?
  |
  +-- Is it a threshold, keyword, prefix, window or weight?
         YES -> it belongs in config.py. (§31 rule 6)
  +-- Is it a rupee amount?
         YES -> almost certainly wrong. Derive it from the measured
                distribution. The median is Rs 899.35. (§31 rule 7)

Am I about to key, join or de-duplicate on Transaction_ID?
  |
  +-- STOP. It is not unique. Use source_row_id. (§4, §27)

Am I about to decide a role or a layer?
  |
  +-- From a single degree, or from hop depth alone?
         YES -> stop. Composite, with a reason string. (§15, §16, §17)
  +-- Am I storing the layer on the account?
         YES -> stop. Layer is per-investigation. (§18)

Am I about to collapse, group or de-duplicate transactions?
  |
  +-- Is it for storage?
         YES -> stop. Parallel edges are the evidence. (§12.2)
  +-- Is it only a display aggregation that can expand to rows?
         YES -> allowed.

Am I about to claim accuracy, precision or a detection rate?
  |
  +-- STOP. There are no labels. There is no ground truth. (§30)
```

41. Data Contract Acceptance Checklist

The contract is satisfied when every line below is true and demonstrable.

#	Criterion	✓
1	Account numbers are strings everywhere; a leading-zero account round-trips CSV → API → UI → PDF unchanged	☐
2	Ingest succeeds with duplicate `Transaction_ID` values present; zero rows dropped	☐
3	`source_row_id` is present on every fact row, edge, evidence object, API transaction payload and cited document line	☐
4	`GET /api/rows/{source_row_id}` returns the verbatim source row	☐
5	`GET /api/transactions/{id}` always returns `matches[]`; the ambiguous case returns `AMBIGUOUS_TRANSACTION_ID` (409)	☐
6	Parallel edges between the same pair are preserved as distinct edges with distinct row ids	☐
7	The profiler publishes measured enums, distributions, bounds and the signal co-occurrence cross-tab	☐
8	No rupee threshold, keyword, IP prefix, device name, window or weight is hard-coded outside `config.py`	☐
9	Every derived feature in §10 is computed exactly once, by one function	☐
10	Money in evidence and documents is `Decimal`, recomputed from cited rows	☐
11	Every displayed number can name its `source_row_ids[]`	☐
12	`dataset_fingerprint` appears in every API response and on every generated document	☐
13	A fingerprint mismatch on stored evidence is rejected with a clear error	☐
14	FIFO attribution matches only strictly earlier inflows; unmatched outflow is reported, not forced	☐
15	`median_hold_seconds` is `null`, never `0`, when it could not be measured	☐
16	No code path sets a role from `in_degree == 0` or `out_degree == 0`	☐
17	Collector and Distributor panels both show inbound and outbound sides	☐
18	`layer` appears only on trace nodes, with `relative_to_victim` and `hop_distance`	☐
19	No global `layer` field exists on the account record	☐
20	Terminal evidence returns `interpretation.verdict: null`	☐
21	Terminal signals alone cannot produce a CRITICAL assessment	☐
22	Every MRI response carries all six components, a reason string, `calibration_regime` and the verbatim disclaimer	☐
23	No accuracy, precision, recall or F1 figure appears anywhere in the product	☐
24	The words "criminal", "guilty" and "confirmed mule" appear nowhere in generated output	☐
25	`net_balance` is labelled as an observed flow residual wherever it is shown	☐
26	The hub guard reports `truncation_reason: "HUB_GUARD"` instead of silently pruning	☐
27	The LLM path is optional, prose-only and validated; disabling it changes no number	☐
28	Ingest + profile + feature build completes within the 60 s budget on the full 2M-row file	☐

42. The Final Rule

> **The data describes what happened.**
>
> **The system describes what the data shows.**
>
> **A human decides what it means.**
>
> Every design decision in ABHEDYA-CHAKRA collapses to keeping those three sentences in that order.

```
WHAT HAPPENED       2,000,000 recorded transfers between 24,873 accounts
                    over 15 days. Immutable. Hashed. Cited by row id.

WHAT IT SHOWS       measured topology, velocity, pass-through, splitting
                    and terminal signals — each explainable, each
                    reproducible from the source rows.

WHAT IT MEANS       a decision by an investigating officer, taken with
                    the evidence in front of them, under their authority
                    and their accountability. Never by this software.
```

The test to apply to any proposed feature:

> If the feature causes the system to state a conclusion a human did not reach, it is out of scope — however impressive it looks in a demo.

---

43. The One-Line Mental Model

```
2 million bank transfers.
No names. No labels. No verdicts.

Money enters at a victim, moves fast through accounts that
collect it, split it and exit it — and every single claim we
make about that movement must point back to a row in the file.
```

Carry one sentence into every code review:

> **"Which source rows prove this?"**

If that question has an answer, the code is probably right. If it does not, the code is making something up — and in a system that produces documents addressed to banks under Section 94 BNSS, making something up is the only unrecoverable failure.

Document Control

Field	Value
Document	ABHEDYA-CHAKRA · Data Context & Data Contract
Version	**1.0**
Status	**Frozen for v1.0**
Date	02 October 2026
Owner	Data & Backend
Supersedes	The data-contract subsection of `ABHEDYA_CHAKRA_PRD.md` v2.1 on `Transaction_ID` uniqueness and duplicate quarantine (§4, §27)
Companion documents	`ABHEDYA_CHAKRA_PRD.md` v2.1 · `docs/BACKEND_API.md` v1.0 · `docs/FRONTEND_UI.md` v1.0
Authority	Highest for **data meaning, data rules and data prohibitions**. Subordinate to the profiler for any measured figure
Associated org	Indore Police Commissionerate
Event	Void Hacks() 8.0

Marker legend

Marker	Meaning
`[PS]`	Stated in or directly derived from the problem statement
`[DEC]`	A design decision taken by the team
`[ASSUME]`	An assumption, recorded so it can be challenged
`[DATA-PRELIM]`	A preliminary measurement — **must be recalculated by the profiler before use**

Change log

Version	Date	Change
1.0	02 October 2026	Initial frozen release. Establishes `source_row_id` as a first-class concept across backend, graph, evidence, APIs and generated documents; supersedes the PRD's `Transaction_ID` uniqueness assertion

Verification status

Claim class	Status
Column names, types and count (11)	Verified against the supplied schema
Row, account, date-range and distribution figures	**[DATA-PRELIM]** — preliminary; the profiler is authoritative
Signal co-occurrence (~2,654 / ~1,327)	**[DATA-PRELIM]** + **[ASSUME]** on the shared-injection reading; resolved by the profiler cross-tab

Claim class	Status
Ground-truth labels	**Confirmed absent.** No accuracy metric is computable
Derived memory and throughput figures	Calculated from the preliminary row/account counts; re-measure after ingest
Legal citations	S.94 BNSS, 2023 (production of documents) verified; S.106 BNSS (seizure/freeze) asserted and flagged for confirmation